

The XRH-1 Hash Function

Siwei Sun, Zhiyu Zhang, Shun Li, Fangyu Zheng,
Yong Liu, Guozhen Liu, Jianhua Wang, Ruixi Lv, Zhen Qin
sunsiwei@ucas.ac.cn
University of Chinese Academy of Sciences, Beijing, China

June 29, 2026

Contents

Chapter 1 Specification	5
1.1 The SPONGE-EDM ^c Construction	5
1.1.1 Construction Definition	5
1.2 The XRH-p Permutation	8
1.2.1 Basic Components	8
1.2.2 Round Function	11
1.3 The XRH-1 Hash Functions	13
Chapter 2 Design Rationale	15
2.1 Design Objectives and Evaluation Axes	15
2.2 Rationale for the SPONGE-EDM Mode	15
2.3 Rationale for the XRH-p Permutation	16
2.4 Nomenclature	16
2.4.1 Technical Perspective: eXtended Resilience Hash	16
2.4.2 Cultural Heritage: Xi Rang Hash	17
2.4.3 Design Synergy	17
2.5 Architectural Positioning	17
Chapter 3 Security Analysis	19
3.1 Security Evaluation of the XRH-p Permutation	19
3.1.1 Differential and Linear Cryptanalysis	19
3.1.2 Impossible Differential and Zero-Correlation Linear Cryptanalysis	21
3.1.3 Zero-sum distinguishers	21
3.2 Security Evaluation of Hash Function XRH-1	22
3.2.1 Provable Security of SPONGE-EDM Family Constructions	22
3.2.2 Randomness Test	23
3.2.3 Differential-based attacks on XRH-1	23
3.2.4 Meet-in-the-Middle attacks on XRH-1	23
3.2.5 Quantum Security of XRH-1	27
3.3 Conclusion	27
Chapter 4 Performance Analysis	29
4.1 Benchmarking Environment	29
4.2 Benchmark Methodology	29
4.3 Performance Results	30
4.4 Discussion	30

Chapter 5 Feature Overview	31
5.1 Innovativeness of the Algorithm	31
5.2 Simplicity of the Algorithm	31
5.3 Flexibility of the Algorithm	31
5.4 Performance Advantages	32
Bibliography	32

Chapter 1

Specification

The hash construction specified in this document is based on the **SPONGE-EDM^c** construction [1], to appear at CRYPTO 2026.

This chapter defines the **SPONGE-EDM^c** construction [1], the permutation **XRH-p**, and the hash functions **XRH-1-n** ($n \in \{512, 768, 1024\}$).

1.1 The **SPONGE-EDM^c** Construction

1.1.1 Construction Definition

Overview. The **SPONGE-EDM^c** construction, which is a member of the **SPONGE-EDM** family [1], is parameterized by:

- Permutation width b ;
- A padding algorithm Pad ;
- Rate parameters r (absorption) and r' (squeezing), with capacity parameters $c = b - r$ (absorption) and $c' = b - r'$ (squeezing);
- Output length $n \in \mathbb{N}$.

The **SPONGE-EDM^c** construction constitutes a family of **SPONGE** variants predicated upon one-way functions. In **SPONGE-EDM^c**, we instantiate one-way function $\mathcal{F}$ as the Encrypted Davies-Meyer construction [2], *but with only c bits of feed-forward* (g, h are permutations):

$$\mathcal{F}(x) = h(g(x) \oplus \lfloor x \rfloor^{(c)}). \quad (1.1)$$

Like the classical **SPONGE** construction, **SPONGE-EDM^c** consists of two main phases: an absorption phase and a squeezing phase. In the absorption phase, message blocks are iteratively injected into the internal state through XORing and then processed by a state-update function $\mathcal{F}$. In the squeezing phase, output blocks are extracted from the final state. The crucial difference from the classical **SPONGE** is the absorption update function: the classical **SPONGE** applies a permutation f directly to the XORed state, while **SPONGE-EDM^c** applies the one-way EDM function $\mathcal{F}(x) = h(g(x) \oplus \lfloor x \rfloor^{(c)})$. The squeezing phase remains essentially the same as in the classical **SPONGE**, typically using the full permutation $f = h \circ g$.

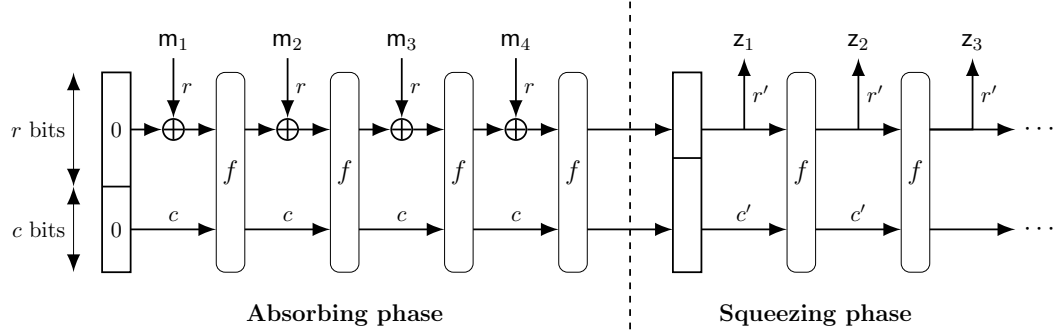
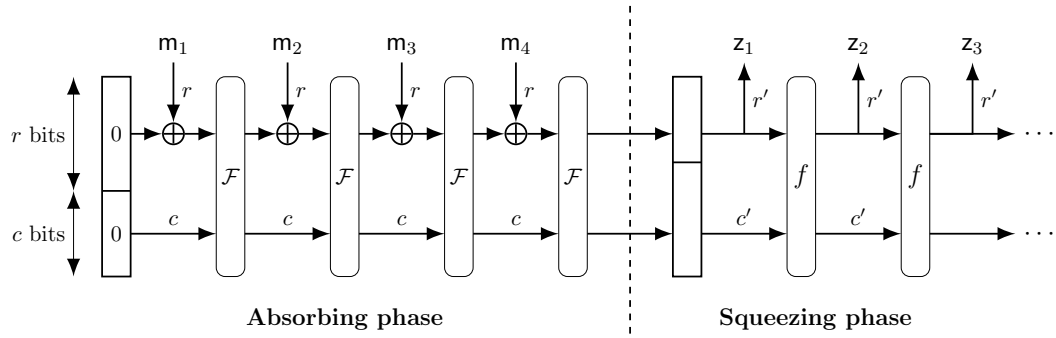


Figure 1.1: The SPONGE construction

Figure 1.2: The SPONGE-EDM^c construction ($\mathcal{F}$ is defined in Eq. 1.1)

Algorithm 1: SPONGE

Input: $\text{msg} \in \mathbb{F}_2^*$, $n \in \mathbb{N}$
Output: n -bit digest $z \in \mathbb{F}_2^n$

```

1  $m \leftarrow \text{msg} \parallel \text{Pad}(r, \|\text{msg}\|_{\mathbb{F}_2})$ 
2  $\alpha \leftarrow \lceil \|\text{msg}\|_{\mathbb{F}_2} / r \rceil$ 
3 Let  $m = (m_1, \dots, m_\alpha) \in (\mathbb{F}_2^r)^\alpha$ 
4  $\text{state} \leftarrow 0^b$ 
5 for  $i$  from 1 to  $\alpha$  do
6    $\text{state} \leftarrow f(\text{state} \oplus (m_i \parallel 0^c))$ 
7 Let  $z \leftarrow \text{Trunc}_{r'}(\text{state})$ 
8 while  $\|z\|_{\mathbb{F}_2} < n$  do
9    $\text{state} \leftarrow f(\text{state})$ 
10   $z \leftarrow z \parallel \text{Trunc}_{r'}(\text{state})$ 
11 return  $\text{Trunc}_n(z)$ 

```

Absorption Phase

Given an input message msg , the absorption process proceeds as follows:

1. Pad the message: $m = \text{msg} \parallel \text{Pad}(r, \|\text{msg}\|_{\mathbb{F}_2})$, resulting in $\alpha = \lceil \|\text{msg}\|_{\mathbb{F}_2} / r \rceil$ blocks of r bits;
2. Parse m as $(m_1, \dots, m_\alpha) \in (\mathbb{F}_2^r)^\alpha$;
3. Initialize the state: $\text{state}_0 = 0^b$;
4. For each block $i = 1, \dots, \alpha$:

$$\text{state}'_{i-1} = \text{state}_{i-1} \oplus (m_i \parallel 0^c), \quad (1.2)$$

$$\text{state}_i = h(g(\text{state}'_{i-1}) \oplus \lfloor \text{state}'_{i-1} \rfloor^{(c)}). \quad (1.3)$$

The output of the absorption phase is the final state state_α .

Squeezing Phase

The squeezing phase remains identical to the classical SPONGE construction. Let $\beta = \lceil n/r' \rceil$ denote the number of output blocks. The digest is generated as:

1. Initialize $\text{state}^{(0)} = \text{state}_\alpha$;
2. $z_1 = \text{Trunc}_{r'}(\text{state}^{(0)})$;
3. For $j = 2, \dots, \beta$:

$$\text{state}^{(j-1)} = f(\text{state}^{(j-2)}), \quad (1.4)$$

$$z_j = \text{Trunc}_{r'}(\text{state}^{(j-1)}), \quad (1.5)$$

$$(1.6)$$

where f is typically instantiated as $h \circ g$ or a full permutation;

4. Output $\text{Trunc}_n(\mathbf{z}_1 \parallel \cdots \parallel \mathbf{z}_\beta)$.

In the **XRH-1**, we adopt the **SPONGE-EDM^c** construction, its complete specification is provided in Algorithm 2.

Algorithm 2: **SPONGE-EDM^c** $[g, h, \text{Pad}, r, r', n]$

Input: Message $\text{msg} \in \mathbb{F}_2^*$
Output: n -bit digest $\mathbf{z}$

```

1 // Padding
2  $\mathbf{m} \leftarrow \text{msg} \parallel \text{Pad}(r, \|\text{msg}\|_{\mathbb{F}_2})$ 
3  $\alpha \leftarrow \|\mathbf{m}\|_{\mathbb{F}_2} / r$ 
4 Parse  $\mathbf{m}$  as  $(\mathbf{m}_1, \dots, \mathbf{m}_\alpha) \in (\mathbb{F}_2^r)^\alpha$ 

5 // Absorption Phase (EDM Mode)
6  $\text{state} \leftarrow 0^b$ 
7 for  $i \leftarrow 1$  to  $\alpha$  do
8    $\text{state} \leftarrow \text{state} \oplus (\mathbf{m}_i \parallel 0^{b-r})$ 
9    $\text{state} \leftarrow h(g(\text{state}) \oplus \lfloor \text{state} \rfloor^{(c)})$            // EDM transformation

10 // Squeezing Phase
11  $\mathbf{z} \leftarrow \text{Trunc}_{r'}(\text{state})$ 
12 while  $\|\mathbf{z}\|_{\mathbb{F}_2} < n$  do
13    $\text{state} \leftarrow f(\text{state})$ 
14    $\mathbf{z} \leftarrow \mathbf{z} \parallel \text{Trunc}_{r'}(\text{state})$ 
15 return  $\text{Trunc}_n(\mathbf{z})$ 
```

1.2 The **XRH-p** Permutation

The **XRH-p** permutation is a 1280-bit permutation designed for efficient software implementation on 64-bit platforms. It adopts the “wide trail strategy” for large state size. The **XRH-p** permutation consists of 18 rounds, each round applying a non-linear layer, a linear layer, and a round constant addition. The non-linear layer is based on a 5-bit almost bent S-box, which provides good resistance against differential and linear cryptanalysis. The linear layer is designed to provide good diffusion properties while being efficient to implement. Overall, the **XRH-p** permutation is designed to be secure against known cryptanalytic attacks while being efficient to implement in software.

1.2.1 Basic Components

State The state size of **XRH-p** is 1280 bits, which can be viewed as a matrix with 20 rows and 64 columns, with row indices from 0 to 19 and column indices from 0 to 63, as shown in Figure 1.3. The i -th row of the state is denoted as $\mathcal{S}[i][\cdot]$, the j -th column of the state is denoted as $\mathcal{S}[\cdot][j]$, and the element at row i and column j is denoted as $\mathcal{S}[i][j]$.

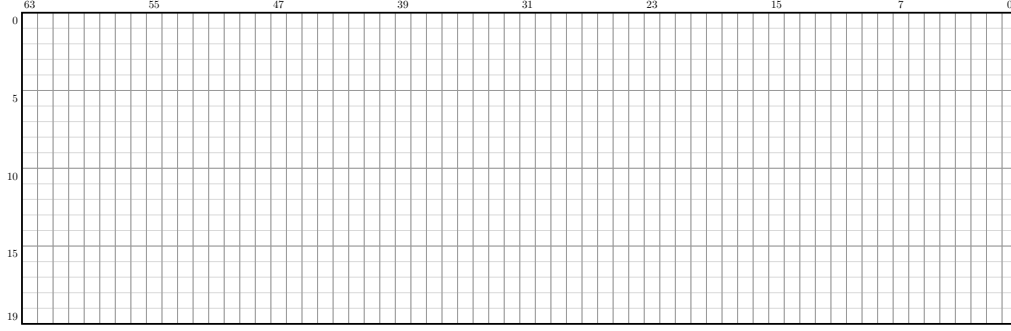


Figure 1.3: State Matrix Representation of XRH-p

S-Box XRH-p uses a 5-bit input, 5-bit output S-box S , defined as follows:

$$S = [4, 14, 10, 5, 9, 24, 20, 0, 18, 28, 30, 21, 2, 23, 29, 13, \\ 22, 27, 15, 7, 26, 12, 16, 3, 8, 1, 19, 31, 25, 11, 17, 6].$$

This S-box is a 5-bit Almost Bent (AB) S-box with differential uniformity of 2 and maximum linear bias of 4. The bitsliced implementation requires 25 operations. The S-box application is illustrated in Figure 1.4, where 4 S-boxes are applied in parallel to each column. Although different colors are used in the figure, the 4 S-boxes are the same S-box S .

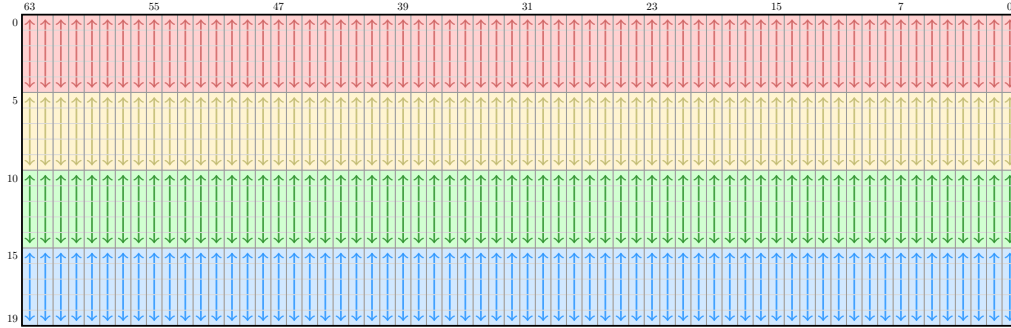


Figure 1.4: S-box Application in XRH-p

MDS Matrix The MDS matrix over $\mathbb{F}_2^5 = \mathbb{F}_2[x]/(x^5 + x^2 + 1)$ is defined as

$$M = \begin{pmatrix} 3 & 2 & 1 & 3 \\ 3 & 1 & 4 & 4 \\ 1 & 3 & 6 & 4 \\ 2 & 2 & 3 & 1 \end{pmatrix}.$$

The XOR gate count for M is 44. The MDS matrix application is illustrated in Figure 1.5, where the MDS matrix is applied to each column. For j -th column $S[\cdot][j]$, we define the

input to the MDS matrix as $[a_0, a_1, a_2, a_3]$, where $a_i = \mathcal{S}[i * 5][j] + 2 \times \mathcal{S}[(i * 5) + 1][j] + 4 \times \mathcal{S}[(i * 5) + 2][j] + 8 \times \mathcal{S}[(i * 5) + 3][j] + 16 \times \mathcal{S}[(i * 5) + 4][j]$, and the output of the MDS matrix as $[b_0, b_1, b_2, b_3]$. Then we have

$$\begin{pmatrix} b_3 \\ b_2 \\ b_1 \\ b_0 \end{pmatrix} = \begin{pmatrix} 3 & 2 & 1 & 3 \\ 3 & 1 & 4 & 4 \\ 1 & 3 & 6 & 4 \\ 2 & 2 & 3 & 1 \end{pmatrix} \begin{pmatrix} a_3 \\ a_2 \\ a_1 \\ a_0 \end{pmatrix}.$$

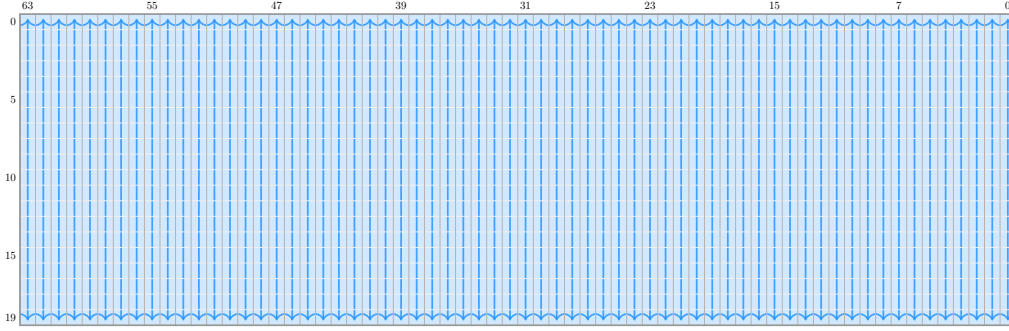


Figure 1.5: MDS Matrix Application in XRH-p

Shift Rows Three different shift row operations are defined, denoted as $\mathcal{SR}_X, \mathcal{SR}_Y, \mathcal{SR}_Z$. They cyclically shift the columns of the state, with the shift amount depending on the row group index $\lfloor i/5 \rfloor$:

- $\mathcal{SR}_X$ partitions the 64 columns into 16 groups of 4 consecutive columns and shifts each group by $\lfloor i/5 \rfloor$ positions:

$$\mathcal{S}[i][j] \mapsto \mathcal{S}[i] \left[4 \left\lfloor \frac{j}{4} \right\rfloor + \left((j \bmod 4) + \left\lfloor \frac{i}{5} \right\rfloor \right) \bmod 4 \right].$$

- $\mathcal{SR}_Y$ partitions the 64 columns into 4 groups of 16 consecutive columns and shifts each group by $4 \lfloor i/5 \rfloor$ positions:

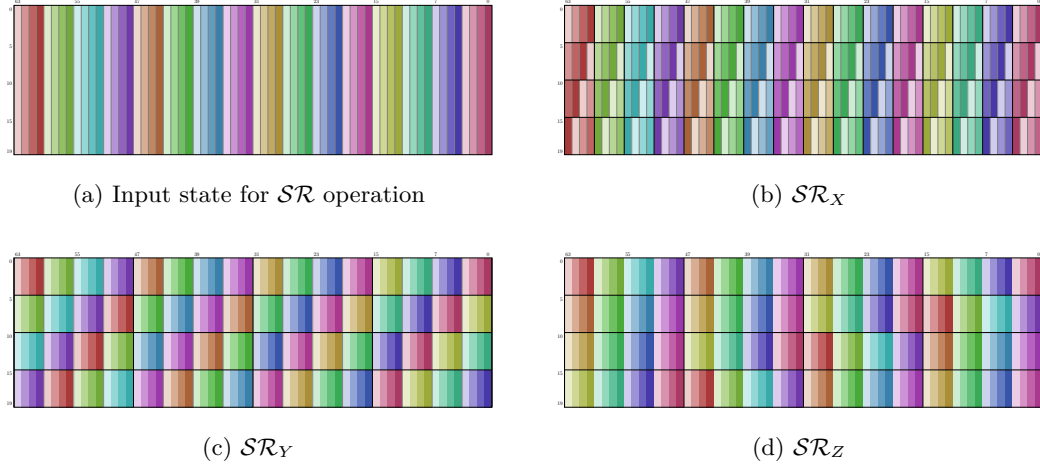
$$\mathcal{S}[i][j] \mapsto \mathcal{S}[i] \left[16 \left\lfloor \frac{j}{16} \right\rfloor + \left((j \bmod 16) + 4 \left\lfloor \frac{i}{5} \right\rfloor \right) \bmod 16 \right].$$

- $\mathcal{SR}_Z$ treats all 64 columns as a single group and shifts them globally by $16 \lfloor i/5 \rfloor$ positions:

$$\mathcal{S}[i][j] \mapsto \mathcal{S}[i] \left[\left(j + 16 \left\lfloor \frac{i}{5} \right\rfloor \right) \bmod 64 \right].$$

Super S-Box A Super S-box is a 20-bit SPS structure built from 4 parallel 5-bit S-boxes and the MDS matrix M . Concretely, the 5-bit S-box S is applied independently to each 5-bit nibble of a 20-bit column chunk; then the MDS matrix M mixes the four resulting nibbles; and finally S is applied again to each nibble. We denote this parallel application of four S-boxes by $S^{\otimes 4}$ and write

$$S' = S^{\otimes 4} \circ M \circ S^{\otimes 4}.$$

Figure 1.6: $\mathcal{SR}$ Operations

Super Mixcolumn The Super Mixcolumn M' is formed by sandwiching the MDS matrix M between a shift-row operation and its inverse, i.e.,

$$M' = \mathcal{SR}^{-1} \circ M \circ \mathcal{SR},$$

where $\mathcal{SR}$ is one of the three shift-row operations $\mathcal{SR}_X, \mathcal{SR}_Y, \mathcal{SR}_Z$ (the same operation is used on both sides). Together, the shift row, the column mixing, and the inverse shift row move bits across 4 adjacent columns, i.e., across four Super S-boxes.

Round Constants The round constants are derived from the binary expansion of the fractional part of π , with a 64-bit round constant being XORed into the most significant 64 bits in each round.

1.2.2 Round Function

The round function is iterated $R = 18$ times; its structure is given in Algorithm 3. Each round consists of the following steps:

1. **Super S-Box layer:** Apply the Super S-box S' to each of the 64 columns.
2. **Super Mixcolumn layer:** Apply the Super Mixcolumn M' to the whole state. This consists of three sub-steps:
 - (a) **Shift row:** Let r be the current round index (starting from 0). Choose the shift-row operation according to $r \bmod 3$: $\mathcal{SR}_X$ if $r \bmod 3 = 0$, $\mathcal{SR}_Y$ if $r \bmod 3 = 1$, and $\mathcal{SR}_Z$ if $r \bmod 3 = 2$ (see Figure 1.6).
 - (b) **Column mixing:** Apply the MDS matrix M to each column.
 - (c) **Inverse shift row:** Apply the inverse of the shift-row operation used in the first sub-step, denoted $\mathcal{SR}_X^{-1}, \mathcal{SR}_Y^{-1}, \mathcal{SR}_Z^{-1}$.
3. **Round constant addition:** XOR the 64-bit round constant RC_r into the first row of the state (i.e., row 0).

Algorithm 3: R-round XRH-p permutation

Input: State $state \in \{0, 1\}^{1280}$
Output: Updated state $state \in \{0, 1\}^{1280}$

```

1 for  $r \leftarrow 0$  to  $R - 1$  do
2   // The following is the non-linear layer of the round function
3   //  $state[5i : 5i + 4][j]$  represents 5 bits from rows  $5i$  to  $5i + 4$ 
   // (inclusive) of column  $j$ 
4   //  $state[*][j]$  represents column  $j$  of the state
5   for  $j \leftarrow 0$  to 63 do
6     for  $i \leftarrow 0$  to 3 do
7        $state[5i : 5i + 4][j] \leftarrow S(state[5i : 5i + 4][j])$  ;           // S-box
8        $state[*][j] \leftarrow MDS(state[*][j])$  ;                         // MDS matrix
9       for  $i \leftarrow 0$  to 3 do
10         $state[5i : 5i + 4][j] \leftarrow S(state[5i : 5i + 4][j])$  ;       // S-box
11    // The following is the linear layer of the round function
12    // Diffusion between 4 Super S-boxes
13    if  $r \bmod 3 = 0$  then
14       $state \leftarrow SR_X(state)$ 
15    else if  $r \bmod 3 = 1$  then
16       $state \leftarrow SR_Y(state)$ 
17    else
18       $state \leftarrow SR_Z(state)$ 
19    for  $j \leftarrow 0$  to 63 do
20       $state[*][j] \leftarrow MDS(state[*][j])$ 
21    if  $r \bmod 3 = 0$  then
22       $state \leftarrow SR_X^{-1}(state)$ 
23    else if  $r \bmod 3 = 1$  then
24       $state \leftarrow SR_Y^{-1}(state)$ 
25    else
26       $state \leftarrow SR_Z^{-1}(state)$ 
27    // Add 64-bit round constant to the first row (row 0)
28     $state[0][*] \leftarrow state[0][*] \oplus RC_r$ 

```

1.3 The XRH-1 Hash Functions

This section specifies the concrete XRH-1 hash function family by instantiating the SPONGE-EDM^c construction from Section 1.1 with the XRH-p permutation from Section 1.2.

Let $b = 1280$ be the permutation width. We use an 18-round permutation split into two 9-round permutations:

$$g = \Pi_{[0,9)}, \quad h = \Pi_{[9,18)}, \quad f = h \circ g = \Pi_{[0,18)}. \quad (1.7)$$

Here $\Pi_{[a,b)}$ denotes the composition of round transformations of XRH-p from round a to round $b - 1$.

For each output length $n \in \{512, 768, 1024\}$, the parameters are set as $c = n + 64$ and $r = b - c$. The squeezing side uses the same rate as absorption, i.e., $r' = r$. Table 1.1 lists both parameter values and per-variant security. The hash function is defined as:

$$\text{XRH-1-}n(\text{msg}) = \text{SPONGE-EDM}^c[g, h, \text{Pad10} * 1, r_n, r_n, n](\text{msg}), \quad (1.8)$$

where $r_n = 1280 - n - 64$, and $n \in \{512, 768, 1024\}$.

Table 1.1: Parameter sets of XRH-1

Variant	b	r	c	Collision	Preimage	2nd preimage
XRH-1-512	1280	704	576	256	512	512
XRH-1-768	1280	448	832	384	768	768
XRH-1-1024	1280	192	1088	512	1024	1024

Padding and Parsing. Given a message $\text{msg} \in \mathbb{F}_2^*$, apply $\text{Pad10} * 1$ at rate r :

$$\mathbf{m} = \text{msg} \parallel 1 \parallel 0^k \parallel 1, \quad (1.9)$$

where $k \geq 0$ is the smallest integer such that $\|\mathbf{m}\|_{\mathbb{F}_2}$ is a multiple of r . Parse $\mathbf{m}$ as $(\mathbf{m}_1, \dots, \mathbf{m}_\alpha)$ with $\mathbf{m}_i \in \mathbb{F}_2^{r_n}$.

The Full Algorithm. Algorithm 4 gives the complete procedure.

Algorithm 4: XRH-1- $n(\text{msg})$

Input: Message $\text{msg} \in \mathbb{F}_2^*$ and output length $n \in \{512, 768, 1024\}$ **Output:** n -bit digest z

```

1  $m \leftarrow \text{msg} \parallel \text{Pad10} * 1$ 
2  $\alpha \leftarrow \|m\|_{\mathbb{F}_2} / r$ 
3 Parse  $m$  as  $(m_1, \dots, m_\alpha) \in (\mathbb{F}_2^r)^\alpha$ 
4  $\text{state} \leftarrow 0^{1280}$ 
5 for  $i \leftarrow 1$  to  $\alpha$  do
6    $\text{state} \leftarrow \text{state} \oplus (m_i \parallel 0^c)$ 
7    $\text{state}' \leftarrow \text{state}$ 
8    $\text{state} \leftarrow g(\text{state})$  // first 9 rounds
9    $\text{state} \leftarrow h(\text{state} \oplus \lfloor \text{state}' \rfloor^{(c)})$  // last 9 rounds and EDM step
10  $z \leftarrow \text{Trunc}_r(\text{state})$ 
11 while  $\|z\|_{\mathbb{F}_2} < n$  do
12    $\text{state} \leftarrow f(\text{state})$  //  $f = h \circ g$ 
13    $z \leftarrow z \parallel \text{Trunc}_r(\text{state})$ 
14 return  $\text{Trunc}_n(z)$ 

```

Chapter 2

Design Rationale

2.1 Design Objectives and Evaluation Axes

This chapter explains why the specification in Chapter 1 uses the SPONGE-EDM^c construction and the XRH-p permutation as the foundation of the XRH-1 family. The goal is not to restate algorithms, but to justify architectural choices and parameter choices in a standards-oriented way.

The design rationale is organized around three evaluation axes:

- **Security margin:** resistance targets for collision, preimage, and second-preimage attacks;
- **Implementation cost:** practical software efficiency under a fixed state width;
- **Specification clarity:** clear composition rules that reduce ambiguity across variants.

From this perspective, the central design question is how to preserve the generic advantages of sponge-based hashing while strengthening absorption behavior for preimage-oriented settings. The answer adopted in this standard is to instantiate sponge absorption with an EDM-style one-way update and to bind that update to a single permutation design that is also used during squeezing.

2.2 Rationale for the SPONGE-EDM Mode

The construction in Section 1.1 keeps the external sponge interface, but modifies the internal absorption update to use an EDM-style one-way transformation. This choice is motivated by the role of inverse computations in generic preimage strategies against classical sponge absorption.

In a classical sponge, absorption and squeezing are both driven by an invertible state permutation. For collision security this is appropriate, but for preimage and second-preimage settings it creates a structural asymmetry: an attacker can often reason about inverse transitions in ways that are irrelevant to the intended use.

By using

$$\mathcal{F}(x) = h(g(x) \oplus \lfloor x \rfloor^{(c)}),$$

absorption is no longer modeled as a plain permutation call. The update mixes the current state with its low-capacity projection before the second permutation stage, which is intended to increase one-way behavior of each absorbed block.

The squeezing process remains sponge-like on purpose. This preserves familiar security analyses for collision-style behavior and keeps the interface compatible with established sponge usage patterns. In this standard, the same permutation design is reused across absorption and squeezing to avoid mode fragmentation.

2.3 Rationale for the XRH-p Permutation

Section 1.2 defines a 1280-bit permutation with 18 rounds. This permutation is selected as a unified primitive for both mode phases in Section 1.3: the first half and second half are used as g and h in absorption, while their composition is used for squeezing.

Wide Trail Strategy. The design of the permutation is based on the Wide Trail strategy [3]. The XRH-p employs 5-bit almost bent S-boxes with desirable cryptographic properties and MDS matrices to ensure strong confusion and diffusion. Its structure is clear and the security is readily analyzable. The permutation marks the first extension of the wide trail strategy to large states exceeding 1024 bits, with carefully selected components that strike a balance between security and performance.

State Width and Software Orientation A 1280-bit state offers a practical middle point between narrow-state designs and very wide-state designs. The width is large enough to support multiple security levels through capacity tuning, while still keeping a software-friendly structure on 64-bit platforms.

Round Structure as a Reusable Core The permutation uses repeated non-linear and linear layers with explicit diffusion operators. The rationale for this structure is composability:

- the same round core can serve as a full permutation for squeezing;
- round splitting naturally provides two sub-permutations for EDM absorption;
- implementation code paths remain aligned across all hash variants.

2.4 Nomenclature

The proposed algorithm is named XRH-1. This nomenclature carries a dual significance, representing both the cultural heritage and the technical objectives of the design. It reflects a commitment to extreme security margins and structural robustness.

2.4.1 Technical Perspective: eXtended Resilience Hash

Technically, XRH serves as an acronym for **eXtended Resilience Hash**, succinctly summarizing the core technical advantages of the primitive: **eXtended Resilience (Structural Robustness)**. The resilience of XRH is derived from two primary design pillars:

1. **Wide Trail Strategy:** Within the 1280-bit permutation, the linear and non-linear layers are meticulously crafted to maximize the lower bound of active S-boxes. This ensures high mathematical resilience against differential and linear cryptanalysis.

2. **SPONGE-EDM Structure:** The integration of the SPONGE-EDM mechanism enhances the structural resilience, improving the security properties compared to standard sponge functions.

2.4.2 Cultural Heritage: Xi Rang Hash

The name “Xi Rang” (息壤) originates from ancient Chinese mythology. It refers to a legendary, self-growing soil that possesses the magical property of expanding infinitely to dam and suppress catastrophic floods.

- **Structural Analogy:** XRH utilizes a variant of the sponge construction. The “self-growing” nature of Xi Rang accurately mirrors the dynamic evolution of the internal state during the *Absorbing* phase, where the message stream is continuously integrated into the large-scale permutation state.
- **Security Symbolism:** In the face of the ever-evolving “floods” of cryptanalytic attacks, XRH is designed to provide an unyielding defense, ensuring that the message digest remains secure and immutable.

2.4.3 Design Synergy

The design philosophy of XRH harmonizes the equilibrium found in Eastern philosophy with the rigorous symmetry of modern block cipher design. By utilizing the Wide Trail Strategy over a 1280-bit state to achieve exhaustive diffusion and the SPONGE-EDM structure to ensure structural integrity, XRH achieves its goal of being as adaptable yet as solid as the legendary “Xi Rang”.

2.5 Architectural Positioning

The design in this standard is positioned as a sponge-family hash architecture with an EDM-style absorption refinement and a dedicated permutation instantiation.

At a high level, the design inherits from sponge practice:

- absorb-then-squeeze interface;
- security interpretation through rate/capacity split;
- variant definition through explicit parameter sets.

Its distinctive choices are:

- EDM-style absorption update instead of plain permutation absorption;
- explicit two-stage permutation split g/h with $f = h \circ g$;
- one deterministic parameter rule $c = n + 64$, $r = 1280 - c$ for all variants.

This chapter focuses on rationale only. Formal security arguments are deferred to Chapter 3, and quantitative benchmark evidence is deferred to Chapter 4.

Chapter 3

Security Analysis

3.1 Security Evaluation of the XRH-p Permutation

In this subsection, the security of XRH-p is evaluated using several classical cryptanalytic techniques, including differential, linear, impossible differential, zero-correlation linear, and integral cryptanalysis.

3.1.1 Differential and Linear Cryptanalysis

Differential and linear cryptanalysis [4, 5] represent two of the most fundamental cryptanalysis techniques. Differential cryptanalysis, first introduced by Biham and Shamir [4], exploits the propagation of differences between two plaintexts through the cryptographic primitive. Let

$$\Delta_i \xrightarrow{p} \Delta_o$$

denote a differential transition of the target cryptographic primitive E , where the transition probability is defined as

$$p = \Pr(C_1 \oplus C_2 = \Delta_o \mid P_1 \oplus P_2 = \Delta_i),$$

and $C_i = E(P_i)$ for $i \in \{0, 1\}$. The effectiveness of differential cryptanalysis relies on constructing differential distinguishers spanning as many rounds as possible while maintaining a sufficiently high transition probability.

In this paper, we employ the MILP-based automated cryptanalysis framework proposed by Sun *et al.* [6] to search for the minimum number of differential active S-boxes of the permutation XRH-p. Our automated search model is described as follows.

Differential Automated Model. Let $x_r^i[j]$ denote the input difference variable of the permutation XRH-p at round r , where $0 \leq i \leq 3$ represents the row index. Each cell consists of 5 bits, with the least significant bit located at the top. Moreover, $0 \leq j \leq 63$ denotes the column index, consistent with Fig. 1.2. In our model, the corresponding difference variables after the first B-box layer, the MixColumn layer, the second S-Box layer, and the Super MixColumn layer are denoted by $y_r^i[j]$, $z_r^i[j]$, $w_r^i[j]$, and $x_{r+1}^i[j]$, respectively. In the following, we describe in detail the differential propagation rules of the S-Box layer, the MixColumn layer, and the Super MixColumn layer.

The S-Box Layer. Since the S-Box is a permutation, the input and output differences remain identical in truncated differentials. Hence, the differential propagation through the S-Box layer satisfies

$$x_r^i[j] = y_r^i[j], \quad z_r^i[j] = w_r^i[j],$$

for $0 \leq i \leq 3, 0 \leq j \leq 63$.

The MixColumn Layer. Since the differential branch number of the MixColumn matrix is 5, the differential propagation through the MixColumn layer satisfies:

$$\begin{aligned} \sum_{0 \leq i \leq 3} (y_r^i[j] + z_r^i[j]) - 5 s_r[j] &\leq 0, \\ y_r^i[j] &\geq s_r[j], \quad 0 \leq i \leq 3, \\ z_r^i[j] &\geq s_r[j], \quad 0 \leq i \leq 3, \end{aligned}$$

for $0 \leq j \leq 63$, where $s_r[j]$ is a binary auxiliary variable.

The Super MixColumn Layer. The Super MixColumn layer differs from the MixColumn layer by the presence of Shift Rows operations before and after the linear transformation. Therefore, its differential propagation rules are similar to those of the MixColumn layer, with the row indices of the difference variables adjusted according to the Shift Rows operation.

Additional Constraints. To avoid trivial solutions, The input difference is required to have at least one active cell. Therefore, the following constraint is added to the model:

$$\sum_{0 \leq i \leq 3} \sum_{0 \leq j \leq 63} x_0^i[j] \geq 1.$$

Objective Function. The objective of our automated model is to minimize the number of differential active S-boxes over R -round **XRH-p**, i.e.,

$$\min \sum_{0 \leq r \leq R-1} \sum_{0 \leq i \leq 3} \sum_{0 \leq j \leq 63} (x_r^i[j] + y_r^i[j]).$$

The minimum number of differential active S-boxes obtained by our automated search is summarized in Table 1. Since the maximum differential probability of the S-box used in **XRH-p** is 2^{-4} , the results indicate that 8-round **XRH-p** are sufficient to resist differential attacks.

Table 3.1: Minimum Number of Differential Active S-boxes N_{DAS} for r -round **XRH-p**

r	1	2	3	4	5	6	7	8
N_{DAS}	5	25	45	125	205	≤ 300	≤ 320	> 320

Linear cryptanalysis, first introduced by Matsui [5], exploits biased linear approximations relating plaintexts and ciphertexts. We use

$$\lambda_i \xrightarrow{q} \lambda_o$$

to denote a linear approximation with correlation

$$q = C(\lambda_i \cdot P \oplus \lambda_o \cdot C),$$

where $C = E(P)$. Due to the well-known duality between differential and linear cryptanalysis, we infer that 8-round XRH-p also provide resistance against linear attacks.

3.1.2 Impossible Differential and Zero-Correlation Linear Cryptanalysis

The impossible differential attack, independently introduced by Biham *et al.* [7] and Knudsen [8], is one of the most powerful cryptanalysis techniques against block ciphers. The core idea of an impossible differential attack is to identify impossible differential distinguishers covering as many rounds as possible whose probability is zero.

In this paper, we adopt the MILP-based automated search models for impossible differential distinguishers independently proposed by Sasaki and Todo [9] and Cui *et al.* [10] to search for impossible differential distinguishers of XRH-p. Compared with the differential automated search model, the impossible differential automated search model

1. The objective function is removed.
2. The following constraints are added to enforce fixed input and output differences:

$$x_0^{i_i}[j_i] = 1, \quad x_R^{i_o}[j_o] = 1.$$

For all $0 \leq i_i, i_o \leq 3$ and $0 \leq j_i, j_o \leq 63$, we independently solve the model for each pair of input and output difference positions. The infeasibility of any instance implies the existence of R -round impossible differential distinguishers. Our experimental results indicate that no 5-round impossible differential distinguisher exists for XRH-p when both the input and output differences contain only one active cell.

Zero-correlation linear cryptanalysis, first introduced by Bogdanov and Rijmen [11], can be regarded as the linear counterpart of impossible differential cryptanalysis and exploits linear approximations whose correlation equals zero. The core idea of a zero-correlation linear attack is to identify zero-correlation linear distinguishers covering as many rounds as possible. Due to the duality between zero-correlation linear cryptanalysis and impossible differential cryptanalysis, we expect that no 5-round zero-correlation linear distinguisher exists for XRH-p when both the input and output masks contain only one active cell.

3.1.3 Zero-sum distinguishers

The degree of the S-box used in XRH-p is 2, and the degree of its inverse is 3. Therefore, the degree of n -round XRH-p is at most 2^n and that of the inverse is at most 3^n , which is the same as SHA-3. It is simple to construct a zero-sum distinguisher for 16-round XRH-p with 10-round forward and 6-round backward, which is only 2 rounds left. Thus, We do not guarantee the absence of full-round zero-sum distinguishers.

The zero-sum distinguishers do not compromise the security claims of XRH-1. The inapplicability to the SPONGE-EDM^c construction stems from the fact that an adversary cannot directly control the inner part of the state. Exploiting a zero-sum distinguisher requires the adversary to choose the input to the permutation freely, which is not possible in the SPONGE-EDM^c construction. During the absorbing phase, the input to the permutation is

the XOR of the previous state and the message block. The adversary can only directly influence the outer part (the rate), while the inner part (the capacity) remains unknown and uncontrolled. Therefore, the adversary cannot realize the specific input sets required by the zero-sum distinguisher, making the distinguisher non-exploitable in any attack on the sponge function itself.

3.2 Security Evaluation of Hash Function XRH-1

3.2.1 Provable Security of SPONGE-EDM Family Constructions

The SPONGE-EDM^c construction achieves the following security bounds in the random permutation model:

Table 3.2: Security bounds of SPONGE-EDM^c (in bits)

Security Notion	Bound	Notes
Indifferentiability	$c/2$	Equivalent to classical sponge
Collision resistance	$\min\{n/2, c/2\}$	Equivalent to classical sponge
Preimage resistance	$\min\{n, c\}$	
Second preimage resistance	$\min\{n, c - \log_2(\alpha)\}$	α : max. message blocks

Preimage Resistance

The key advantage of SPONGE-EDM^c is the elimination of the $c/2$ barrier for (second) preimage resistance. The construction achieves full n -bit preimage resistance because:

- The EDM transformation $\mathcal{F}(x) = h(g(x) \oplus \lfloor x \rfloor^{(c)})$ is a one-way function;
- Generic preimage attacks on the classical sponge rely on computing f^{-1} during absorption;

Second Preimage Resistance

Second preimage resistance is improved from $\min\{n, c/2\}$ to $\min\{n, c - \log_2(\alpha)\}$, where α denotes the maximum number of message blocks in the first preimage. For practical parameters (e.g., $\alpha \leq 2^{64}$):

- With $c = 576$: achieves approximately 512-bit second preimage resistance;
- With $c = 1088$: achieves approximately 1024-bit second preimage resistance.

Collision Resistance and Indifferentiability

Remarkably, the strengthening of (second) preimage resistance does not compromise other security properties. SPONGE-EDM^c maintains:

- The same $\min\{n/2, c/2\}$ collision resistance bound as the classical sponge;
- The same $c/2$ -bit indifferentiability bound from a random oracle.

These bounds are tight and match the generic attacks on the sponge construction, which do not rely on the invertibility of the absorption function. For a detailed security proof, please refer to the accompanying paper. [1]

3.2.2 Randomness Test

XRH-1 have passed the randomness tests conducted by the NIST Statistical Test Suite (STS). The tests were performed on 100 samples of 1,000,000 hash output bits generated from counter inputs. The results indicate that the output of XRH-1 exhibits strong randomness properties.

3.2.3 Differential-based attacks on XRH-1

Based on the differential cryptanalysis results of the permutation presented in Section 3.1.1, the proposed algorithm offers sufficient resistance against differential-based attacks, including but not limited to differential collision attacks and rebound attacks.

3.2.4 Meet-in-the-Middle attacks on XRH-1

In this paper, we employ the MILP-based modeling technique proposed by Bao *et al.* [12, 13] to construct Meet-in-the-Middle attacks on the SPONGE-EDM mode and the SPONGE-DM mode. In the following, we first present the detailed MILP modeling framework for the Meet-in-the-Middle cryptanalysis.

Notations and Encoding. We encode the attributes of each state cell according to whether its value is determined by neutral bits in the forward direction, the backward direction, both directions, or neither. Each cell is represented by two binary variables (x, y) .

- **Gray:** The cell value is a predefined constant and therefore known and fixed in both the forward and backward chunks.

$$(x, y) = (1, 1).$$

- **Blue:** The cell value is determined by forward neutral bytes together with predefined constants. Hence, it is known and active in the forward chunk but remains unknown in the backward chunk.

$$(x, y) = (1, 0).$$

- **Red:** The cell value is determined by backward neutral bytes together with predefined constants. Hence, it is known and active in the backward chunk but remains unknown in the forward chunk.

$$(x, y) = (0, 1).$$

- **White:** The cell value depends simultaneously on both forward and backward neutral bytes and thus cannot be computed independently in either chunk.

$$(x, y) = (0, 0).$$

Let $(x_{0,r}^i[j], x_{1,r}^i[j])$ denote the input variable of the permutation **XRH-p** at round r , where $0 \leq i \leq 3$ represents the row index. Each cell consists of 5 bits, with the least significant bit located at the top. Moreover, $0 \leq j \leq 63$ denotes the column index, consistent with Fig. 1.2. In our model, the corresponding variables after the first B-box layer, the MixColumn layer, the second S-Box layer, and the Super MixColumn layer are denoted by $(y_{0,r}^i[j], y_{1,r}^i[j])$, $(z_{0,r}^i[j], z_{1,r}^i[j])$, $(w_{0,r}^i[j], w_{1,r}^i[j])$, and $(x_{0,r+1}^i[j], x_{1,r+1}^i[j])$, respectively. In the following, we describe in detail the variable propagation rules of the S-Box layer, the MixColumn layer, and the Super MixColumn layer.

The S-Box Layer. The S-Box operation does not change the attribute of the cells, thus is not involved in our model.

The MixColumn Layer. The attribute propagation through the MixColumn layer follows the rules below.

1. If any input cell in a column is **White**, then all output cells of this column become **White**.
2. The **Gray** attribute is preserved without consuming degrees of freedom only when all input cells of the column are **Gray**.
3. If no **White** cell exists in the input column, a column containing i **Blue**, j **Red**, and k **Gray** cells propagates to an output column consisting of i' **Blue**, j' **Red**, k' **Gray**, and ℓ' **White** cells. This propagation consumes $j' + k'$ degrees of freedom from **Blue** cells and $i' + k'$ degrees of freedom from **Red** cells, where

$$i + j + k = i' + j' + k' + \ell' = 4,$$

and

$$\begin{cases} j' + k' < i \leq 4, & \text{if } i \neq 0, \\ j' + k' = 4, & \text{otherwise,} \end{cases} \quad \begin{cases} i' + k' < j \leq 4, & \text{if } j \neq 0, \\ i' + k' = 4, & \text{otherwise.} \end{cases}$$

Let a_i^I be a binary variable indicating whether the i -th input cell is **White**. Furthermore, we introduce two integer variables c_x and c_y representing the degrees of freedom consumed from **Blue** and **Red** cells, respectively. To describe the global attribute of the input column, we introduce three auxiliary binary variables:

$$a = \begin{cases} 1, & \text{if a **White** cell exists in the column,} \\ 0, & \text{otherwise,} \end{cases}$$

$$b = \begin{cases} 1, & \text{if all cells are **Gray**,} \\ 0, & \text{otherwise,} \end{cases} \quad c = \begin{cases} 1, & \text{if no **Blue** cell exists,} \\ 0, & \text{otherwise.} \end{cases}$$

The corresponding propagation constraints are formulated as a set of linear inequalities:

$$\begin{cases}
\sum_{j=0}^3 y_{0,r}^i[j] - 4b \geq 0, \\
\sum_{j=0}^3 y_{0,r}^i[j] - b \leq 3, \\
\sum_{j=0}^3 y_{1,r}^i[j] - 4c \geq 0, \\
\sum_{j=0}^3 y_{1,r}^i[j] - c \leq 3.
\end{cases}$$

$$\begin{cases}
\sum_{i=0}^3 z_{0,r}^i[j] + 4a \leq 4, \\
\sum_{i=0}^3 z_{1,r}^i[j] + 4a \leq 4, \\
\sum_{i=0}^3 (y_{0,r}^i[j] + z_{0,r}^i[j]) - 5b \leq 3, \\
\sum_{i=0}^3 (y_{0,r}^i[j] + z_{0,r}^i[j]) - 8b \geq 0, \\
\sum_{i=0}^3 (y_{1,r}^i[j] + z_{1,r}^i[j]) - 5c \leq 3, \\
\sum_{i=0}^3 (y_{1,r}^i[j] + z_{1,r}^i[j]) - 8c \geq 0.
\end{cases}$$

$$\begin{cases}
\sum_{i=0}^3 z_{1,r}^i[j] - c_x \geq 0, \\
\sum_{i=0}^3 z_{1,r}^i[j] - 4c - c_x \leq 0, \\
\sum_{i=0}^3 z_{1,r}^i[j] + 4c + c_x \leq 8, \\
\sum_{i=0}^3 z_{0,r}^i[j] - c_y \geq 0, \\
\sum_{i=0}^3 z_{0,r}^i[j] - 4b - c_y \leq 0, \\
\sum_{i=0}^3 z_{0,r}^i[j] + 4b + c_y \leq 8.
\end{cases}$$

The Super MixColumn Layer. The Super MixColumn layer differs from the MixColumn layer by the presence of Shift Rows operations before and after the linear transformation. Therefore, its propagation rules are similar to those of the MixColumn layer, with the row indices of the difference variables adjusted according to the Shift Rows operation.

The Matching Rule. We consider two matching strategies. In the first strategy, the matching point is placed at the Super MixColumn layer of the last round. The input of the matching corresponds to the input of the Super MixColumn layer, while the output is defined as the first $\min\{r, p\}$ squeezed output bits (the SPONGE-EDM mode), or the XOR between the input $\min\{r, p\}$ bits and the first $\min\{r, p\}$ squeezed output bits (the SPONGE-DM mode). In the second strategy, the input and output of the matching point are defined as the input $\min\{r, p\}$ bits and the first $\min\{r, p\}$ squeezed output bits, respectively.

The first matching strategy. The cells at the matching point are required to satisfy the following conditions:

1. Both **Blue** and **Red** cells appear in the input and output columns,
2. The sum of **Blue**, **Red**, and **Gray** cells in the input and output columns is no less than 5.

Therefore, the system of inequalities modeling the first matching strategy can be formulated as follows:

$$\begin{cases} dom_r[j] = 1, & \text{if } \sum_{0 \leq i \leq 3} (\beta_r^i[j] + \beta_0^i[j]) \geq 5, \\ dom_r[j] = 0, & \text{otherwise,} \end{cases}$$

$$\begin{cases} \beta_0^i[j] = 1, & \text{if } z_{0,0}^i[j] + z_{1,0}^i[j] \geq 1, \\ \beta_0^i[j] = 0, & \text{otherwise.} \end{cases}$$

$$\begin{cases} \beta_r^i[j] = 1, & \text{if } z_{0,r}^i[j] + z_{1,r}^i[j] \geq 1, \\ \beta_r^i[j] = 0, & \text{otherwise.} \end{cases}$$

$$Dom = \sum_{0 \leq j \leq 63} dom_r[j].$$

The second matching strategy. The input and output cells at the matching point are required to be either a **Red** cell and a **Blue** cell, respectively, or one being a **Blue/Red** cell and the other being a **Gray** cell. Therefore, the system of inequalities modeling the first matching strategy can be formulated as follows:

$$\begin{cases} dom_r^i[j] = 1, & \text{if } x_{0,r+1}^i[j] + x_{0,0}^i[j] = 1 \text{ and } x_{1,r+1}^i[j] + x_{1,0}^i[j] = 1, \\ dom_r^i[j] = 1, & \text{if } x_{0,r+1}^i[j] + x_{0,0}^i[j] + x_{1,r+1}^i[j] + x_{1,0}^i[j] = 3, \\ dom_r^i[j] = 0, & \text{otherwise,} \end{cases}$$

$$Dom = \sum_{0 \leq i \leq 3} \sum_{0 \leq j \leq 63} dom_r^i[j].$$

Objective Function. Denote by $\vec{d}_b$ and $\overleftarrow{d}_r$ the essential degrees of freedom of the forward and backward computations after propagation, respectively. The objective of the search model is to maximize a variable τ_{Obj} , subject to the following constraints:

$$\begin{cases} \tau_{\text{Obj}} & \leq \vec{d}_b, \\ \tau_{\text{Obj}} & \leq \overleftarrow{d}_r, \\ \tau_{\text{Obj}} & \leq \text{Dom}. \end{cases}$$

Our experiments demonstrate that the best Meet-in-the-Middle attacks found for both the SPONGE-EDM mode and the SPONGE-DM mode reach 6 rounds.

3.2.5 Quantum Security of XRH-1

The security analysis in the previous subsections focused on the classical setting. With the rapid development of quantum computing, it is also necessary to evaluate the security of XRH-1 against known quantum attacks. According to the requirements of the Next-generation Commercial Cryptographic Algorithms Program, submitted hash algorithms must resist known classical and quantum attacks, and the complexity of any quantum attack should be no lower than that of generic quantum algorithms.

Generic quantum attacks on hash functions. The most relevant quantum algorithms for hash function cryptanalysis are Grover’s search algorithm and quantum collision-search algorithms.

- **Preimage and second-preimage attacks.** Grover’s algorithm [14] finds a preimage of an n -bit hash value with approximately $2^{n/2}$ quantum queries, which is optimal in the quantum query model. Consequently, the quantum security against preimage and second-preimage attacks is roughly half the output length.
- **Collision attacks.** The quantum collision search proposed by Brassard, Høyer, and Tapp [15] achieves a time complexity of approximately $2^{n/3}$, but it requires a comparable amount of quantum memory. Subsequent analyses have shown that, once realistic quantum memory costs are taken into account, the effective complexity of quantum collision attacks is $2^{2n/5}$ [16].

Based on the classical security analysis of XRH-1 in the preceding sections and the acceleration offered by quantum attacks, we conclude that XRH-1 is sufficiently resistant to quantum attacks. XRH-1 satisfies the requirement that the algorithm must resist known classical and quantum attacks, and that the complexity of quantum attacks must be no lower than that of generic quantum algorithms.

3.3 Conclusion

In this chapter, we have provided a comprehensive security evaluation of the XRH-p permutation and the XRH-1 hash function using both automated cryptanalysis frameworks and provable security bounds.

The security of the underlying XRH-p permutation was rigorously tested against several classical cryptanalytic techniques. By employing MILP-based automated search models, we determined the following:

- **Differential and Linear Cryptanalysis:** The permutation achieves a sufficiently high number of active S-boxes to resist these attacks within 8 rounds. Specifically, the minimum number of differential active S-boxes exceeds 320 for an 8-round configuration, ensuring that the differential probability remains well below the security threshold.
- **Impossible Differential and Zero-Correlation Linear Cryptanalysis:** Our analysis confirms that no 5-round distinguishers exist for **XRH-p** when both the input and output differences/masks contain only one active cell, providing a robust security margin for the full-round design.

Regarding the **XRH-1** hash function and its structural construction:

- **Provable Security:** We demonstrated that the **SPONGE-EDM^c** construction maintains the standard indistinguishability and collision resistance of the classical sponge while significantly improving preimage and second preimage resistance. By eliminating the $c/2$ barrier, the construction achieves full n -bit preimage resistance.
- **Structural Attacks:** Using an MILP-based modeling technique for Meet-in-the-Middle (MitM) attacks, we evaluated both the **SPONGE-EDM^c** mode and the **SPONGE-DM^c** mode. The results indicate that the best MitM attacks found are limited to 6 rounds, further validating the security of the overall sponge construction against sophisticated algebraic and structural cryptanalysis.
- **Quantum Security:** Based on our security analysis of the **XRH-p** permutation and the **SPONGE-EDM^c** mode, we believe that **XRH-1** is resistant to all known quantum attacks, including Grover's algorithm and quantum collision-search algorithms.

In conclusion, the results presented in this chapter indicate that **XRH-1** offers a high level of security against both classical and quantum adversaries. The combination of the wide-trail design of the **XRH-p** permutation and the enhanced properties of the **SPONGE-EDM^c** construction ensures that the hash function is resistant to all known cryptanalytic threats under the proposed parameters.

Chapter 4

Performance Analysis

This chapter presents the performance evaluation of the proposed hash function instances. The evaluation follows the self-assessment guidelines for the Next-generation Commercial Cryptographic Algorithms Program on the x86 architecture.

4.1 Benchmarking Environment

All measurements were performed on a single x86-64 core with the configuration listed in Table 4.1.

Table 4.1: Performance evaluation platform

Item	Description
CPU model	Intel(R) Core(TM) Ultra 7 265H
CPU frequency	3.7 GHz
Architecture	x86-64
Operating system	Linux 6.6.87.2-microsoft-standard-WSL2
Compiler	GCC 13.3.0

4.2 Benchmark Methodology

The performance test program uses the `CryptHash` interface declared in the algorithm-instance header file. For each algorithm instance, the following procedure is applied:

- **Message lengths.** Eight standardized input sizes S1–S8 are used: 32, 128, 512, 1024, 4096, 8192, 16384, and 65536 bytes.
- **Iteration count.** Each level is executed at least 100 times. The iteration count is chosen so that the total processed volume per level is approximately 32 MiB.
- **Warm-up.** 32 preliminary calls are performed before timing to reduce cache and branch-prediction cold-start effects.
- **Timing.** Wall-clock time is measured with `clock_gettime(CLOCK_MONOTONIC)`. CPU cycles are captured with the x86 RDTSC instruction fenced by `LFENCE`.

- **Metrics.** The reported metrics are cycles per byte (cpb).

The reference implementations are compiled with `gcc -std=c99 -Wpedantic -Wall -Wextra -O2`.

4.3 Performance Results

Table 4.2 presents the cycles-per-byte values of the three reference implementations for all S1–S8 message lengths. This view highlights how each algorithm scales as the message length increases and how the digest size influences the per-byte cost.

Table 4.2: Reference implementation cycles-per-byte across S1–S8

Instance	S1	S2	S3	S4	S5	S6	S7	S8
XRH-1-512	47.543	23.149	17.510	17.605	16.731	16.915	16.327	16.474
XRH-1-768	93.677	45.488	31.632	29.445	27.144	26.630	26.382	26.286
XRH-1-1024	321.371	124.524	77.431	68.334	62.855	61.998	61.724	61.220

4.4 Discussion

Scalability with message length. All instances exhibit the expected decay of cycles-per-byte as the message length increases. The transition from short messages (S1, S2) to medium-length messages (S3, S4) is rapid, and the asymptotic regime is essentially reached by S5–S6 (4–8 KiB). This indicates that the per-call setup cost is small compared with the per-block processing cost.

Effect of digest size. Within each family, the cycles-per-byte grows with the digest length. For XRH-1, the asymptotic cpb rises from approximately 16.5 for 512-bit output to 26.3 for 768-bit output and to 61.2 for 1024-bit output.

Impact of Rate on Throughput. The XRH-1 family keeps the permutation width fixed at $b = 1280$ bits and varies the capacity (and therefore the rate) with the output length. Because every absorbed message block triggers an EDM update involving the full 18-round permutation, a smaller rate directly translates into more permutation calls per byte.

Short-Message Penalty. At $\ell = 32$ B the cpb is dominated by the fixed per-call cost. XRH-1-1024 pays a particularly high penalty (≈ 321 cpb) because its narrow 192-bit rate. By contrast XRH-1-512 does not need extra call, yielding roughly 48 cpb. This confirms that the rate choice has a strong impact not only on bulk throughput but also on latency-sensitive use cases such as hashing of short packet headers or nonce material.

Chapter 5

Feature Overview

In this chapter, we explicitly declare the features of the submitted **XRH-1** hash function family. The algorithm is carefully designed to balance security and efficiency, with its core features categorized into innovativeness, simplicity, flexibility, and implementation performance.

5.1 Innovativeness of the Algorithm

The primary innovation of **XRH-1** lies in its underlying mode of operation, the **SPONGE-EDM^c** construction. Unlike classical sponge functions that use a standard invertible permutation for absorption, **XRH-1** integrates an Encrypted Davies-Meyer (EDM) style one-way transformation. This architectural shift breaks the structural symmetry and fundamentally eliminates the classical $c/2$ barrier for preimage and second-preimage resistance. As a result, the hash function achieves full n -bit preimage resistance while strictly preserving the standard collision and indistinguishability bounds of the traditional sponge.

5.2 Simplicity of the Algorithm

XRH-1 is built upon a highly unified and elegant architecture. It utilizes a single reusable cryptographic core, the 18-round **XRH-p** permutation. The simplicity is evident in the permutation's explicit round splitting: the first 9 rounds (g) and the last 9 rounds (h) serve as two sub-permutations for the EDM absorption phase, while their exact composition ($f = h \circ g$) is reused for the squeezing phase. Furthermore, **XRH-1** adopts a single, deterministic parameter rule across all its variants ($c = n + 64$, $r = 1280 - c$). This avoids fragmented code paths and ensures clean, unified, and easily verifiable implementations.

5.3 Flexibility of the Algorithm

The 1280-bit state width of the **XRH-p** permutation provides a highly flexible foundation. The algorithm naturally supports multiple output digest lengths ($n \in \{512, 768, 1024\}$) by simply tuning the rate (r) and capacity (c) parameters. This allows users and implementers to easily navigate the security-performance trade-off curve. The design scales smoothly from high-throughput variants (e.g., **XRH-1-512**) to ultra-high-security variants (e.g., **XRH-1-1024**)

within the exact same structural framework without requiring any changes to the underlying primitive.

5.4 Performance Advantages

XRH-1 is explicitly designed and optimized for efficient software implementation, particularly on modern 64-bit platforms. The state size and wide-trail strategy are tailored for software-friendly execution. Its non-linear layer utilizes a 5-bit almost bent S-box that requires only 25 operations in a bitsliced implementation, which minimizes the computational overhead. Practical benchmark results on an Intel x86_64 processor demonstrate excellent performance: the **XRH-1-512** variant achieves an asymptotic throughput of approximately 16 CPU cycles per processed byte (cpb) for bulk hashing. Even the high-security **XRH-1-1024** variant maintains a practical asymptotic throughput of roughly 61 cpb, proving its efficiency across different performance requirements.

Bibliography

- [1] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance - application to hash-based signature schemes. *IACR Cryptol. ePrint Arch.*, 2025:963, 2025.
- [2] Benoît Cogliati and Yannick Seurin. EWCDM: an efficient, beyond-birthday secure, nonce-misuse resistant MAC. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part I*, volume 9814 of *Lecture Notes in Computer Science*, pages 121–149. Springer, 2016.
- [3] Joan Daemen and Vincent Rijmen. AES and the wide trail design strategy. In Lars R. Knudsen, editor, *Advances in Cryptology - EUROCRYPT 2002, International Conference on the Theory and Applications of Cryptographic Techniques, Amsterdam, The Netherlands, April 28 - May 2, 2002, Proceedings*, volume 2332 of *Lecture Notes in Computer Science*, pages 108–109. Springer, 2002.
- [4] Eli Biham and Adi Shamir. Differential cryptanalysis of des-like cryptosystems. *J. Cryptol.*, 4(1):3–72, 1991.
- [5] Mitsuru Matsui. Linear cryptanalysis method for DES cipher. In Tor Helleseth, editor, *Advances in Cryptology - EUROCRYPT '93, Workshop on the Theory and Application of Cryptographic Techniques, Lofthus, Norway, May 23-27, 1993, Proceedings*, Lecture Notes in Computer Science, pages 386–397. Springer, 1993.
- [6] Siwei Sun, Lei Hu, Peng Wang, Kexin Qiao, Xiaoshuang Ma, and Ling Song. Automatic security evaluation and (related-key) differential characteristic search: Application to simon, present, lblock, DES(L) and other bit-oriented block ciphers. In Palash Sarkar and Tetsu Iwata, editors, *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014. Proceedings, Part I*, Lecture Notes in Computer Science, pages 158–178. Springer, 2014.
- [7] Eli Biham, Alex Biryukov, and Adi Shamir. Cryptanalysis of skipjack reduced to 31 rounds using impossible differentials. *J. Cryptol.*, 18(4):291–311, 2005.
- [8] Lars Knudsen. Deal-a 128-bit block cipher. 1998.
- [9] Yu Sasaki and Yosuke Todo. New impossible differential search tool from design and cryptanalysis aspects - revealing structural properties of several ciphers. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications*

- of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part III*, Lecture Notes in Computer Science, pages 185–215, 2017.
- [10] Tingting Cui, Keting Jia, Kai Fu, Shiyao Chen, and Meiqin Wang. New automatic search tool for impossible differentials and zero-correlation linear approximations. *IACR Cryptol. ePrint Arch.*, 2016:689, 2016.
 - [11] Andrey Bogdanov and Vincent Rijmen. Linear hulls with correlation zero and linear cryptanalysis of block ciphers. *Des. Codes Cryptogr.*, 70(3):369–383, 2014.
 - [12] Zhenzhen Bao, Xiaoyang Dong, Jian Guo, Zheng Li, Danping Shi, Siwei Sun, and Xiaoyun Wang. Automatic search of meet-in-the-middle preimage attacks on aes-like hashing. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part I*, Lecture Notes in Computer Science, pages 771–804. Springer, 2021.
 - [13] Zhenzhen Bao, Jian Guo, Danping Shi, and Yi Tu. Superposition meet-in-the-middle attacks: Updates on fundamental security of aes-like hashing. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part I*, Lecture Notes in Computer Science, pages 64–93. Springer, 2022.
 - [14] Lov K. Grover. A fast quantum mechanical algorithm for database search. In Gary L. Miller, editor, *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, pages 212–219. ACM, 1996.
 - [15] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In Claudio L. Lucchesi and Arnaldo V. Moura, editors, *LATIN '98: Theoretical Informatics, Third Latin American Symposium, Campinas, Brazil, April, 20-24, 1998, Proceedings*, volume 1380 of *Lecture Notes in Computer Science*, pages 163–169. Springer, 1998.
 - [16] André Chailloux, María Naya-Plasencia, and André Schrottenloher. An efficient quantum collision search algorithm and implications on symmetric cryptography. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part II*, volume 10625 of *Lecture Notes in Computer Science*, pages 211–240. Springer, 2017.